Optimal Abstractions for Learnable Activation Functions

Michael Shell
School of Electrical and
Computer Engineering
Georgia Institute of Technology
Atlanta, Georgia 30332–0250

Homer Simpson
Twentieth Century Fox
Springfield, USA
Email: homer@thesimpsons.com

James Kirk
and Montgomery Scott
Starfleet Academy
San Francisco, California 96678–2391
Telephone: (800) 555–1212

***Abstract*—We present a generalized framework for the formal verification of neural networks featuring any form of arbitrary non-linear activation functions. By constructing optimal piecewise affine (PWA) abstractions of these functions, our method establishes strict local error bounds that are then propagated through the network to enable sound property verification via Mixed Integer Linear Programming (MILP). A fundamental challenge in PWA approximations is the trade-off between abstraction precision and verification complexity, where excessive linear pieces yield intractable MILP formulations. To overcome this, we propose an optimized dynamic programming (DP) algorithm that systematically computes the optimal PWA abstraction for general activation functions, guaranteeing tighter output bounds. By explicitly combining this per-layer DP formulation with a network-wide knapsack optimization, our framework minimizes the necessary global piece count footprint while strictly adhering to an pre-agreed error tolerance. Crucially, our *architecture-agnostic* approach is broadly applicable to diverse networks consisting of non-linear activations, including standard Multi-Layer Perceptrons (MLPs) and emerging architectures like Kolmogorov-Arnold Networks (KANs). We perform extensive empirical evaluations on real world datasets and learning tasks to demonstrate that the upfront cost of our DP algorithm is vividly outweighed by its ability to secure substantially tighter verification bounds.**

I. Introduction

II. Related Work

III. Preliminaries

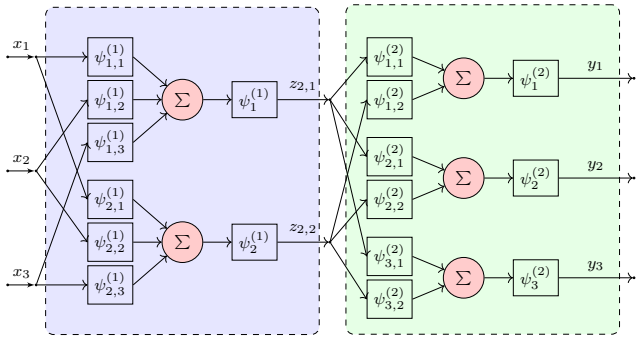

Fig. 1: A two-layer KAN with three inputs $\mathbf{x} = (x_1, x_2, x_3)$ and three outputs $\mathbf{y} = (y_1, y_2, y_3)$, i.e., $K = 2$ with $n_1 = 3$, $n_2 = 2$, $n_3 = 3$. Each layer is shaded differently.

Multi-Layer Perceptrons (MLPs) are typically used with fixed activation functions that are non-linear and applied element-wise across the nodes of a layer which can be easily approximated with piecewise affine functions. For example, in a standard MLP with K layers, the output of the i^{th} layer is given by $\Phi_i(\mathbf{z}_i) = \sigma(W_i\mathbf{z}_i + b_i)$, where σ is the activation function (typically $tanh$, sigmoid, or ReLU), W_i is the weight matrix and b_i is the bias vector for layer i.

Kolmogorov-Arnold Networks (KAN) [1] on the other hand, are a more general class of networks that allow for learnable activation functions. KANs are inspired by the well-known Kolmogorov-Arnold representation theorem in real-analysis that states that any continuous function $f : \mathbb{R}^n \to \mathbb{R}$ can be expressed as a composition of univariate functions and addition [2], [3]. Therefore, KANs represent multivariate continuous functions in terms of additions of univariate representations [4], [1]. There have been numerous approaches to defining KANs. Our formalism is based on the recent work of [1] that uses multiple layers involving polynomial splines. For simplicity of presentation, we will consider KANs with just one output. However, our overall approach extends easily to KANs with multiple outputs.

Definition 1 (Kolmogorov-Arnold Networks (KAN))**.** *A KAN N (Fig. 1) over inputs $\mathbf{x} : (x_1, \ldots, x_n)$ with $K > 0$ layers is a function of the form: $f_N(\mathbf{x}) = \Phi_K(\Phi_{K-1}(\ldots\Phi_1(\mathbf{x})\ldots))$, wherein each Φ_i is a function $\mathbb{R}^{n_i} \to \mathbb{R}^{n_{i+1}}$ that represents the i^{th} layer, and is written as follows: $\Phi_i(\mathbf{z}_i) = (\phi^{(i)}_1(\mathbf{z}_i), \ldots, \phi^{(i)}_{n_{i+1}}(\mathbf{z}_i))$, wherein $\mathbf{z}_i \in \mathbb{R}^{n_i}$ represents the inputs for layer i. Finally, each $\phi^{(i)}_j : \mathbb{R}^{n_i} \to \mathbb{R}$ has a Kolmogorov-Arnold representation of the form: $\phi^{(i)}_j(\mathbf{z}_i) = \psi^{(i)}_{j,0}\left(\sum_{k=1}^{n_i} \psi^{(i)}_{j,k}(\mathbf{z}_{i,k})\right)$. The univariate functions $\psi^{(i)}_{j,k}$ are assumed to satisfy Assumption 1.*

Assumption 1 (Assumptions on ψ)**.** *We will assume that ψ satisfies the following:*

1) *Bounded Domain: There is a constant $L > 0$ such that $\psi(z) = 0$ for $z \notin [-L, L]$.*
2) *$\psi(z)$ is continuous and differentiable over $(-L, L)$.*
3) *There is a procedure* DerivativeInterval(ψ, a, b) *that given input interval $z \in [a, b]$ wherein $-L < a < b < L$, outputs an interval $[\ell, u]$ such that $[\ell, u] \supseteq$*

https://doi.org/10.xxxx/xxxxx

$\left\{\frac{d\psi}{dz} \mid z \in [a,b]\right\}$. *Also, there exists an error tolerance* $\epsilon > 0$ *such that: a)* $\ell \leq \min_{z\in[a,b]} \frac{d\psi}{dz} \leq \ell + \epsilon$, *b)* $u - \epsilon \leq \max_{z\in[a,b]} \frac{d\psi}{dz} \leq u$.

These assumptions can be satisfied for commonly used univariate functions in KANs including B-Splines [1], [5], Padé approximations [6], Chebyshev polynomials [7], Gaussian processes [8], and Fourier Series [9].

a) Range Analysis Problem: : The range analysis problem seeks a guaranteed range for the outputs of a network given a set of inputs. Let $f_N : \mathbb{R}^n \to \mathbb{R}$ be a KAN with K layers as defined in Def. 1. Suppose we define a range $\mathbf{x} \in [\ell, u]$, our goal is to find a "tight" range that over-approximates all the possible output values. In other words, we wish to find $\alpha, \beta \in \mathbb{R}$ with $\alpha \leq \beta$ such that

$$\forall\ \mathbf{x} \in [\ell, u], f_N(\mathbf{x}) \in [\alpha, \beta] \tag{1}$$

We guarantee that the range output by our algorithm is "tight": i.e, for a given $\delta > 0$,

$$\alpha + \delta \geq \min_{\mathbf{x}\in[\ell,u]} f_N(\mathbf{x}),\ \text{and}\ \max_{\mathbf{x}\in[\ell,u]} f_N(\mathbf{x}) \geq \beta - \delta \tag{2}$$

I.e., the bounds computed by our approach are no more than δ away from the tightest possible bounds.

Problem 1 (Range Verification Problem)**.** *Given a KAN* N *computing a function* $f_N : \mathbb{R}^n \to \mathbb{R}$*, an input range* $\mathbf{x} \in [\ell, u]$ *and a tightness parameter* $\delta > 0$*, we wish to find a range over the output* $[\alpha, \beta]$ *satisfying Eqs.* (1) *and* (2)*.*

We can extend Problem 1 to cases wherein the inputs $\mathbf{x}$ range over a polyhedron $P[\mathbf{x}]$ since our approach is based on a MILP formulation.

IV. Optimal Piecewise Affine (PWA) Approximations for Univariate Functions

Let us consider a univariate function $\psi(z) = \psi^{(i)}_{j,k}(z)$, which appears as part of a KAN in Fig. 1. Given a univariate function $\psi(z)$ for $z \in [-L, L]$ satisfying Assumption 1, we wish to approximate it as a continuous PWA function $\widehat{\psi}(z)$ with at most k pieces, where $k \geq 2$ is an user input, while minimizing the error: $\mathsf{e}(\psi) = \max_{z\in[-L,L]} \left|\psi(z) - \widehat{\psi}(z)\right|$.

A k-piece univariate, continuous PWA function is represented by a sequence of $k-1$ breakpoints: $\langle z_1, \ldots, z_{k-1}\rangle$ that satisfy the ordering constraint (for convenience, we assume $z_0 = -L$ and $z_k = L$) where, $-L = z_0 < z_1 < \cdots \leq z_{k-1} < L = z_k$ such that, the PWA function $\widehat{\psi}$ is defined as:

$$\widehat{\psi}(z) = \psi(z_i) + (z - z_i)\frac{\psi(z_{i+1}) - \psi(z_i)}{z_{i+1} - z_i},\ \text{if} z \in [z_i, z_{i+1}]$$

where, $i \in \{0, \ldots, k-1\}$ and $\widehat{\psi}(z) = 0$ if $z \leq -L$ or $z \geq L$. Furthermore, we will discretize the continuous range $z \in [-L, L]$ into $j_{\max} + 1$ intervals using grid points that are $\Delta = \frac{2L}{j_{\max}}$ distance apart, so that each grid point is of the form $z_j = -L + j\Delta$ for a natural number $j \in \mathbb{N}$. The optimal PWA problem is stated as follows:

Problem 2 (Optimal Univariate PWA Approximation)**.** *The problem of optimally approximating* $\psi(z)$ *is as follows:*

Inputs: *An algorithm for* ψ*, limits* $-L, L$*, discretization factor* Δ *and limit on number of pieces* k*.*

Output: *Breakpoints* $z_1 \leq z_2 \leq \cdots \leq z_{k-1}$ *such that the discretized error* $e(\Delta) = \max_{z\in\{-L,-L+\Delta,-L+2\Delta,\cdots,L\}} \left|\psi(z) - \widehat{\psi}(z)\right|$ *is minimized.*

Note that if two breakpoints z_j, z_{j+1} *coincide in the output, we treat them as the single breakpoint.*

Let $\mathsf{n}(j;\Delta) = -L + j\Delta$. The optimal univariate PWA approximation can be solved using a dynamic programming algorithm first proposed by Bellman et al [10], [11]. The algorithm defines a value function: $V(\hat{j}, j, \hat{k})$ for the optimal discretization error for a PWA approximation of ψ over the range $[\mathsf{n}(\hat{j}), \mathsf{n}(j_{\max})]$ such that the points in the range $[\mathsf{n}(\hat{j}), \mathsf{n}(j)]$ belong to the same piece, for natural numbers $\hat{j} < j \leq j_{\max}$ and $\hat{k} \leq k$. The function V is recursively defined as

$$V(\hat{j}, j, \hat{k}) = \begin{cases} \mathsf{sPE}(\psi, \hat{j}, j_{\max}), & \text{if } \hat{k} > 0 \\ \infty, & \text{if } \hat{k} \leq 0 \\ \min(\max(\mathsf{sPE}(\psi, \hat{j}, j), & \\ \qquad V(j, j+1, \hat{k}-1)), & \\ \quad V(\hat{j}, j+1, \hat{k})), & \text{otherwise} \end{cases} \tag{3}$$

The function $\mathsf{sPE} = \mathsf{singlePieceError}(\psi, j_1, j_2)$ is defined measures the error between $\psi(z)$ and the line joining the points $(\mathsf{n}(j_1), \psi(\mathsf{n}(j_1)))$ and $((\mathsf{n}(j_2), \psi(\mathsf{n}(j_2)))$, over the discretized interval $[\mathsf{n}(j_1), \mathsf{n}(j_2)]$. Let s_{j_1,j_2} be the slope of the line $\frac{\psi(\mathsf{n}(j_2)) - \psi(\mathsf{n}(j_1))}{\mathsf{n}(j_2) - \mathsf{n}(j_1)}$.

$$\max_{z\in\{\mathsf{n}(j_1),\ldots,\mathsf{n}(j_2)\}} \left|\psi(z) - (\psi(\mathsf{n}(j_1)) + (z - \mathsf{n}(j_1))s_{j_1,j_2})\right| .$$

The dynamic programming formulation upon memoization runs in time $O(j^3_{\max}k)$ assuming that evaluations of $\psi(z)$ and other arithmetic operations are $O(1)$. The optimal error equals the value $V(0, 1, k)$ and the solution can be recovered in $O(j^2_{\max}k)$ steps by traversing the table. Let $\widehat{\psi}$ be the function obtained as a result of running Bellman's algorithm.

a) Discretization vs. Continuous Error:: We consider the gap between the discretized error $e(\Delta) = \max_{z\in\{-L,-L+\Delta,-L+2\Delta,\cdots,L\}} \left|\psi(z) - \widehat{\psi}(z)\right|$ and error over the entire continuum of values: $e(\widehat{\psi}, \psi) = \max_{z\in[-L,L]} |\psi(z) - \widehat{\psi}(z)|$. We will derive a correction term that can be used to account for this gap. This correction can be used to modify the singplePieceError calculation used in Eq. (3). The derivation of this error is detailed in Appendix A. Furthermore, this provides a way to control Δ to a "small enough" value so that the discretization error falls below a user-provided limit.

b) Trade-Off Table:: Finally, we use the dynamic programming formulation to yield a tradeoff table that has the following useful information for a given function $\psi(z)$. Let $k_{\max}$ be an upper limit on the number of breakpoints. The

tradeoff table stores, for each value $k \in [1, k_{\max}]$, the minimum error $\mathsf{e}(\psi, k)$ obtained by running Bellman's algorithm seeking a function $\widehat{\psi}_k$ with at most k pieces and information on the optimal breakpoints. Given the full memo-table for a run of Bellman's algorithm with $k = k_{\max}$, we can read off the value $V(0, 1, k)$ for $k \in [1, k_{\max}]$.

V. Optimal PWA for Entire KAN

We will now consider how to approximate the entire KAN into a PWA function, while ensuring that the overall error $\max_{\mathbf{x}} |f_N(\mathbf{x}) - \hat{f}_N(\mathbf{x})| \leq \delta$ for a fixed δ. Let us suppose we replace each unit in the KAN $\psi_{j,k}^{(i)}$ by a PWA function $\widehat{\psi}_{j,k}^{(i)}$ incurring an error $\mathsf{e}_{j,k}^{(i)}$. We will derive a bound for the entire KAN for the value $\max_{\mathbf{x}\in\mathbb{R}^n} |f_N(\mathbf{x}) - \hat{f}_N(\mathbf{x})|$. First, we prove an error propagation bound for each unit $\psi_{j,k}^{(i)}$ under inputs $z, \hat{z}$.

Lemma 1. *For each unit $\psi_{j,k}^{(i)}$, there exists a constant $\mathsf{L}_{j,k}^{(i)} \geq 0$ such that all $z, \hat{z}$,*

$$|\psi_{j,k}^{(i)}(z) - \widehat{\psi}_{j,k}^{(i)}(\hat{z})| \leq \mathsf{e}_{j,k}^{(i)} + \mathsf{L}_{j,k}^{(i)}|z - \hat{z}|$$

Proof. Let $\psi = \psi_{j,k}^{(i)}$, $\widehat{\psi} = \widehat{\psi}_{j,k}^{(i)}$, $\psi'_{\max} = \max_{z\in[-L,L]} \left|\frac{d\psi}{dz}\right|$ and $\mathsf{e} = \mathsf{e}_{j,k}^{(i)}$. We may write

$$|\psi(z) - \widehat{\psi}(\hat{z})| \leq |\psi(z) - \psi(\hat{z})| + |\psi(\hat{z}) - \widehat{\psi}(\hat{z})| \leq \psi'_{\max}|z - \hat{z}| + \mathsf{e}$$

We obtain the result by setting $\mathsf{L}_{j,k}^{(i)} = \psi'_{\max}$ □

We will now state the main result that casts the overall error of each output of the KAN in terms of the approximation error $\mathsf{e}_{j,k}^{(i)}$ at each unit. Let $y = f_N(\mathbf{x})$ and $\hat{y} = \hat{f}_N(\mathbf{x})$.

Theorem 1. *For each unit $\psi_{j,k}^{(i)}$ there exists a constant $W_{j,k}^{(i)} \geq 0$ such that the overall approximation error is $|y - \hat{y}| \leq \sum_{i=1}^{K} \sum_{j=1}^{n_{i+1}} \sum_{k=1}^{n_i} W_{j,k}^{(i)} \times \mathsf{e}_{j,k}^{(i)}$.*

Furthermore, for any i, j, k, the constant $W_{j,k}^{(i)}$ is:

$$W_{j,k}^{(i)} = \sum_{\pi\in\Pi_{j,k}^{(i)}} \prod_{\psi_{j',k'}^{(i')}\in\pi} \mathsf{L}_{j',k'}^{(i')}, \textit{wherein},$$

$\Pi_{j,k}^{(i)}$ denotes the set of all paths from the output of the unit $\psi_{j,k}^{(i)}$ to the output of the KAN. The constant $W_{j,k}^{(i)}$ is the sum over all paths of the product of the Lipschitz constants $\mathsf{L}_{j',k'}^{(i')}$ (from Lemma 1) of the units encountered along each path.

Proof is provided in Appendix B

A. Optimal KAN Abstraction as Knapsack

Theorem 1 shows how to calculate the error once we have decided upon a PWA abstraction of each unit $\psi_{j,k}^{(i)}$ yielding the error term $\mathsf{e}_{j,k}^{(i)}$. The constants $W_{j,k}^{(i)}$ corresponding to each unit can be computed by traversing the graph corresponding to the neural network starting from nodes in the first layer and moving on to the subsequent layers. The key problem is therefore to decide how many pieces to allocate to each unit so that (a) the total error of the piecewise affine approximation is within the desired worst case error bound δ; and (b) the sum over the number of pieces for each unit for the entire KAN is minimized. We note that the sum of pieces is the correct objective to minimize since this number will determine the complexity of the verification problem, especially the MILP formulation that we will provide in Section VI. We formulate the problem of optimal abstraction as a variant of the well-known knapsack problem called the multi-option knapsack problem [12].

Problem 3 (Multi-Choice Knapsack Problem)**.** *An instance of the multi-choice knapsack problem is defined as follows:*

Inputs: *We have $N \geq 1$* options*, wherein each option is a set of $k \geq 1$ items that are available to be chosen and the items belonging to two different options are disjoint. Furthermore, for each option i, we have a table of non-negative weights (listed in ascending order) and values for the items in option i:*

Item	$I_1^{(i)}$	$I_2^{(i)}$	$\cdots$	$I_k^{(i)}$
Weights	$w_1^{(i)} <$	$w_2^{(i)} <$	$\cdots$	$< w_k^{(i)}$
Value	$v_1^{(i)}$	$v_2^{(i)}$	$\cdots$	$v_k^{(i)}$

We are given a total weight budget $W \geq 0$. We will assume feasibility*: i.e, $\sum_{i=1}^{N} w_1^{(i)} \leq W$.*

Output: *Choose a set of N items, choosing precisely one item from each option such that (a) the total weight of chosen items remains within the budget W and (b) the total value of the chosen items is maximized.*

Theorem 2. *The (decision version) of the multi-option knapsack problem is **NP**-complete.*

Proof is provided in Appendix C. Choosing an optimal abstraction for a KAN reduces to a multi-option knapsack problem: (a) Each option corresponds to a single unit $\psi_{j,k}^{(i)}$ and thus, there are as many options as the number of units in the KAN; (b) For each option corresponding to the unit $\psi_{j,k}^{(i)}$, each item corresponds to a choice of the number of pieces $p_{j,k}^{(i)} \in [1, k_{\max}]$ for the piecewise linearization $\widehat{\psi}_{j,k}^{(i)}$, the weight is given by the product $W_{j,k}^{(i)} \times \mathsf{e}_{j,k}^{(i)}$, wherein $\mathsf{e}_{j,k}^{(i)}$ is obtained from the trade-off table discussed in Section IV, and the value for each item is given by the number $-p_{j,k}^{(i)}$, so that more pieces in the linearization has less value. (c) The weight budget W is the same as the total error budget δ.

Let us assume a multi-option knapsack problem instance with N options, each having k items. Further, the weights $w_j^{(i)}$ and the budget W are all natural numbers. The dynamic programming formulation is based on a value function that we will denote $\mathsf{MO}(i, \hat{W})$ that asks for the optimal selection of items considering options i until N (inclusive) and with a weight budget $\hat{W}$.

$$\mathsf{MO}(i, \hat{W}) = \begin{cases} \infty \text{ if } \hat{W} < 0 \\ 0 \text{ if } i > N \ \wedge \ \hat{W} \geq 0 \\ \min_{j=1}^{k} \left(v_j^{(i)} + \mathsf{MO}\left(i + 1, \hat{W} - w_j^{(i)}\right)\right) \end{cases}$$

The optimal answer is obtained by computing $\mathsf{MO}(1, W)$ through memoization.

Theorem 3. *The dynamic programming algorithm for multi-option knapsack runs in time $O(WNk)$.*

In the setting of optimal KAN abstractions, we will assume that every error $\mathsf{e}_{j,k}^{(i)}$ is a whole number multiple of some small constant c and the total error budget $\delta = W \times c$. The running time complexity is linear in the network size, since N is the number of units in the network.

VI. Mixed Integer Linear Programming Encoding

In this section, we will sketch a Mixed Integer Linear Programming (MILP) formulation for computing the output ranges for a piecewise linearized KAN while incorporating the error bounds to yield a result that is valid for the original nonlinear KAN model. The MILP formulation encodes the semantics of each piecewise linearized unit using a combination of binary and real valued variables.

Let $\widehat{\psi}$ be a piecewise linearized model corresponding to the unit ψ in the KAN with the error $\max_z |\widehat{\psi}_{j,k}^{(i)} - \psi_{j,k}^{(i)}| = \mathsf{e}$. Assume that $\widehat{\psi}$ has ℓ pieces:

$$\widehat{\psi}(z) = \begin{cases} 0 & z \leq -L \\ a_i z + b_i & z \in [z_i, z_{i+1}],\ i \in \{0, \ldots, \ell - 1\} \\ 0 & z \geq L \end{cases}$$

Suppose z denotes the real-valued input to ψ and y denotes the (real-valued) output. We will add a vector of $\ell + 2$ binary variables $\mathbf{w} = (w_{-1}, \ldots, w_\ell)$ wherein w_{-1} will be 1 if $z \leq -L$, $w_i = 1$ if $z \in [z_i, z_{i+1}]$ and $w_\ell = 1$ if $z \geq L$.

First, we enforce that the input can belong to just one interval: $w_{-1} + w_0 + \cdots + w_\ell = 1$. Let $M_z > 0$ be a large enough number such that the input z to the unit satisfies $|z| \leq M_z$. The estimation of M_z will be detailed in Appendix D.

We encode the implication $w_{-1} = 1 \Rightarrow\ z \leq -L$ using the "M-trick" [13] i.e. $z \leq -Lw_{-1} + M_z(1 - w_{-1})$. The remaining constraints over z are as follows:

$$\begin{aligned} &z_i w_i - M_z(1 - w_i) \ \leq\ z \ \leq\ z_{i+1} w_i + M_z(1 - w_i) \ \leftarrow \\ &\text{Big-M trick} : w_i = 1 \ \Rightarrow\ z \in [z_i, z_{i+1}] \\ &z \ \geq\ L w_\ell - M_z(1 - w_\ell) \ \leftarrow \\ &\text{Big-M trick} : w_\ell = 1 \ \Rightarrow\ z \geq L \end{aligned}$$

We now consider constraints on the output y. Let M_y be such that $\forall z\ |\psi(z)| \leq M_y$. The estimation of M_y is based on the known function $\hat{\psi}$ and error bound e, and is also explained in Appendix D.

The constraints on the output include:

$$\begin{aligned} &0w_{-1} - M_y(1 - w_{-1}) \leq y \leq 0w_{-1} + M_y(1 - w_{-1}) \quad \leftarrow \\ &\text{Big-M trick} : w_{-1} = 1 \ \Rightarrow\ y = 0 \\ &a_i z + b_i - \mathsf{e} - (2M_y)(1 - w_i) \\ &\leq y \leq a_i z + b_i + \mathsf{e} + (2M_y)(1 - w_i) \\ &0w_\ell - M_y(1 - w_\ell) \leq y \leq 0w_\ell + M_y(1 - w_\ell) \end{aligned}$$

We note that the MILP encoding of each unit yields constraints that relate its outputs to the inputs. The overall constraint for a KAN combines that for each unit, while ensuring that the set of binary variables used for various units are distinct. The encoding of summation nodes is straightforward.

VII. Results

To this end, we run experiments on four types of KAN (*Small*, *Medium*, *Medium-2*, and *Large*) networks with increasing number of parameters. We then compare the output tightness of our method (*Optimized*) with the baseline (*Vanilla*) approach. KANs are implemented using the FastKAN library [14]. Although this architecture closely resembles the original KAN implementation, it significantly accelerates training with a negligible performance tradeoff, as noted by the authors. Specifically, FastKAN employs radial basis functions (RBFs) alongside learnable spline activations; each layer computes RBFs, applies spline weights to construct univariate functions, and optionally integrates base linear terms with layer normalization.

We consider two approaches: (a) *Vanilla* the baseline method that uses dynamic programming to find the minimax-optimal segment placement at a fixed count k applied uniformly to all splines and (b) *Optimized* uses the same dynamic programming approach but solves an ILP that minimizes total approximation error subject to a global segment budget $B = k \cdot |\mathcal{S}|$, where $|\mathcal{S}|$ is the total number of splines in the network. In this paper we show the results on four model families, and the rest of the results are available in the Appendix E.

In our experiments we consider ℓ_∞-balls given by $\mathcal{B}_\infty(\mathbf{x}, r) = \{\mathbf{x}' \mid \|\mathbf{x} - \mathbf{x}'\|_\infty \leq r\}$ with radii $r \in \{0.05, 0.1, 0.25, 0.5, 1.0, 2.0, 5.0\}$ and we sweep segment counts $k \in \{1, 5, 10, 20, 30, 40, 50\}$. Our implementation is based on Python 3.11 and uses Gurobi as the MILP and ILP solver. Spline abstraction and segment allocation are parallelized via `joblib`. All experiments are run with a fixed input domain centered at the origin and with fixed random seeds for reproducibility.

The hyperparameters are shared across all the experiments. Primarilty we have set the maximum segments $S_{\max} = 50$, MIP gap $\epsilon_{\text{gap}} = 0.15$, and per-output-neuron MILP timeout $T = 500$s. For each spline, the DP table is computed at resolution $\max(50, 2k)$ with sample points drawn from 1000 high-resolution curve evaluations spanning the RBF grid plus two extrapolation bins. Finally, all Lipschitz constants are estimated from those high-resolution samples and used to propagate sensitivity weights backward through the network for ILP-weighted error allocation. For the input-width sweep under the *Optimized* method, the segment allocation is fixed using target error $\delta = 1.5 \cdot \delta_{\min}$, where $\delta_{\min} = \sum_s \min_k \tilde{e}_s(k)$ is the sum over splines of the minimum achievable Lipschitz-weighted approximation error.

Overarching questions: We now describe the evaluation method on the benchmarks and ask the following two overarching questions: *(1) does the Optimized approach yield tighter output bounds than the Vanilla baseline at equal segment budgets? what happens as the segment budget increases? (2) does our approach scale as model parameters grow?*

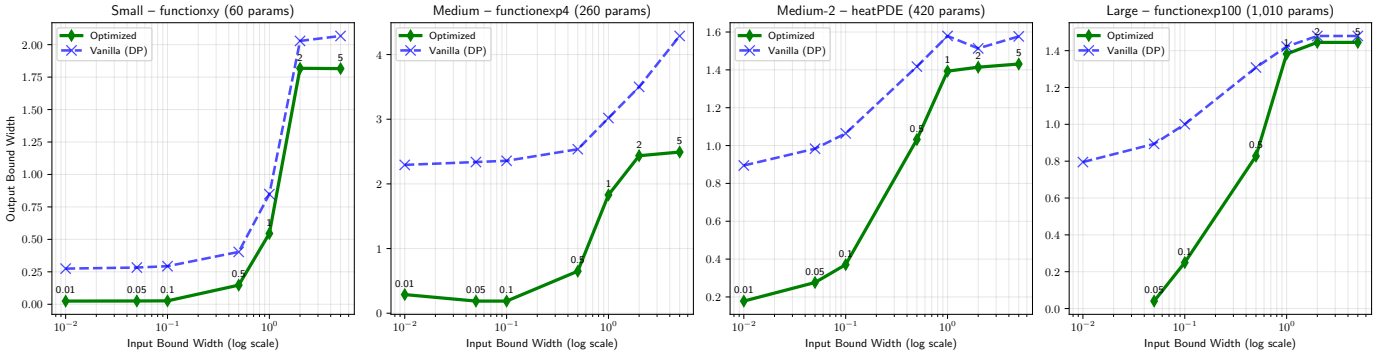

Fig. 2: Tightness of the methods compared by showing how the output bound width changes as input width is varied. We see that across all methods, the output bound width increases as the input bound width increases. However, our method (*Optimized*) consistently achieves tighter bounds than the baseline (*Vanilla*) across all input widths. {Small: [2, 2, 1] KAN; Medium: [4, 4, 2, 1] KAN; Medium-2: [2, 10, 2, 1] KAN; Large: [100, 1, 1] KAN; (x params): x trainable parameters per KAN}

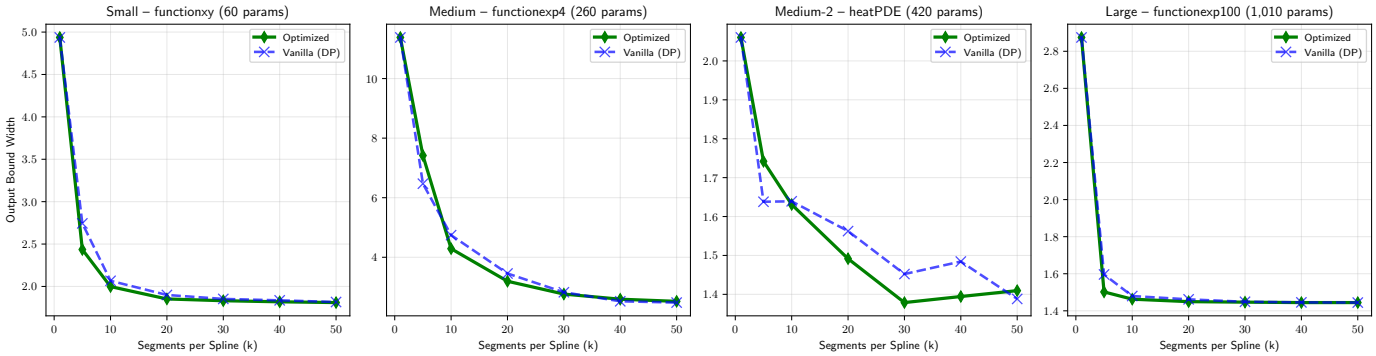

Fig. 3: Tightness of the methods compared by showing how the output bound width changes as segment count per spline is varied. We see that across all methods, the output bound width increases as the segment count increases. Our method (*Optimized*) consistently achieves tighter bounds than the baseline (Vanilla) across all segment counts. {Small: [2, 2, 1] KAN; Medium: [4, 4, 2, 1] KAN; Medium-2: [2, 10, 2, 1] KAN; Large: [100, 1, 1] KAN; (x params): x trainable parameters per KAN}

Learning tasks: We consider a simple but robust task of learning three special functions and a partial differential equations (PDEs) using KANs. These examples are taken from the benchmarks of the original paper [1] and consist of four distinct learning scenarios. The four tasks are:

1) *Small*: `funcxy`: $f(x, y) = xy$ (2 inputs; 60 parameters; [2, 2, 1] KAN)
2) *Medium*: `funcexp4`: $f(x_1, x_2, x_3, x_4) = \exp(\frac{1}{2}(\sin(\pi(x_1^2 - x_2^2)) + \sin(\pi(x_3^2 + x_4^2))))$ (4 inputs; 260 parameters; [4, 4, 2, 1] KAN)
3) *Medium-2*: `heatPDE`: $u_t = \nu u_{xx}$, $[x, t] \in [0, 1] \times [0, 1]$, $\nu = 0.01$ with Dirichlet boundary and sinusoidal initial conditions. (2 inputs; 420 parameters; [2, 10, 2, 1] KAN)
4) *Large*: `funcexp100`: $f(x_1, \ldots, x_{100}) = \exp(\frac{1}{100}\sum_{i=1}^{100}\sin^2(\frac{\pi x_i}{2}))$ (100 inputs; 1010 parameters; [100, 1, 1] KAN)

The size of the KANs are again determined by the best settings as reported in the original paper, and the training is done such that the models achieve a mean squared error (MSE) of less than 10^{-4} on a held-out test set.

*a) **Q1: Bound Tightness — Optimized vs. Uniform Allocation**:* By varying the input perturbation radius and segment budget, we can evaluate the ablative tightness of the proposed method. Figure 2 evaluates robustness under varying input perturbation radii. The advantage of optimized allocation is most pronounced in the small-radius regime, precisely where tight verification bounds are operationally most valuable. For example, we see that the *Small* benchmark at $r = 0.005$ shows the optimized abstraction reduces the output width from 0.275 to 0.025, corresponding to an $11\times$ improvement. The gains are even larger on *Medium, Medium-2* and *Large* ($22\times$ at $r = 0.025$) networks. This behavior is consistent with the structure of interval error propagation i.e. for small perturbation sets, approximation error accumulates primarily along a small number of highly sensitive spline paths, allowing the ILP allocator to concentrate segments where they maximally reduce propagated uncertainty, whereas uniform allocation expends budget indiscriminately across low-impact splines.

On the contraty, as the perturbation radius increases the verified bounds produced by both methods gradually tend to converge. In this regime, global nonlinear effects dominate local

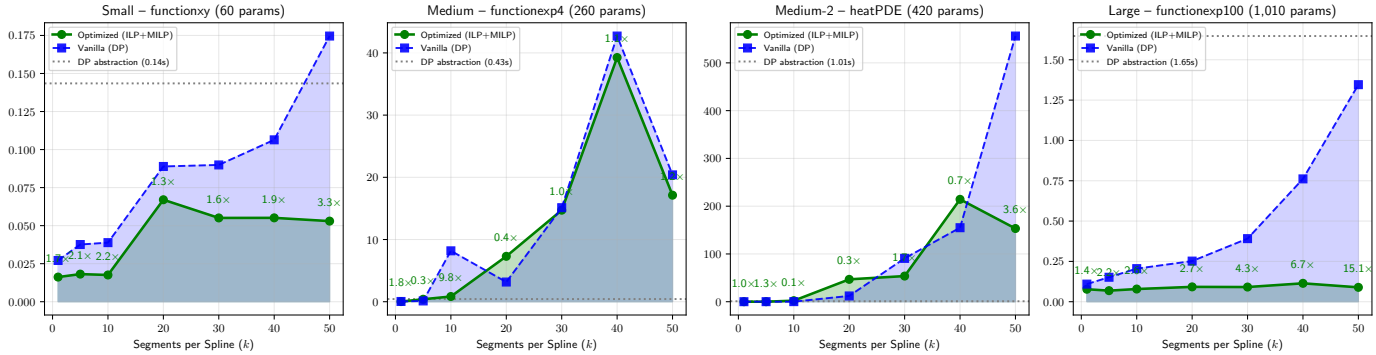

Fig. 4: Timing summary for the different methods. We see that our method (Optimized) is less computationally expensive than the baseline (Vanilla) and also achieves tighter bounds. {Small: [2, 2, 1] KAN; Medium: [4, 4, 2, 1] KAN; Medium-2: [2, 10, 2, 1] KAN; Large: [100, 1, 1] KAN; (x params): x trainable parameters per KAN}

approximation error. We additionally observe that, for the *Large* benchmark at $r = 0.001$, the optimized verifier returns ∞. In this case, the target approximation constraint $\delta = 1.5 \times \delta_{\min}$ is infeasible under the prescribed segment budget, indicating that no admissible PWL abstraction satisfying the target error exists without increasing $S_{\max}$.

Figure 3 further reports verified output bound width as a function of the segment budget k, where *Vanilla* assigns k segments uniformly to every spline and *Optimized* allocates the equivalent global budget $B = k \cdot |\mathcal{S}|$ through the ILP formulation. At $k = 1$, both methods coincide by construction, since a single segment per spline eliminates any allocation degrees of freedom. For increasing budgets, the optimized allocation yields systematically tighter abstractions. On the *Small* benchmark, the optimized verifier achieves a 13% reduction in output width at $k = 5$ (2.44 versus 2.74), while the gap contracts to below 1% as k increases, reflecting convergence of both abstractions toward the underlying nonlinear function as the PWL approximation saturates.

Finally, the *Large* benchmark exhibits the same asymptotic behavior but reaches saturation substantially earlier. Here, the local spline approximation error is already small at modest budgets due to the smoothness of f, limiting the benefit of adaptive allocation to approximately. At low budgets, local spline sensitivities may fail to capture higher-order interactions between propagated approximation errors, leading to suboptimal allocation decisions.

b) ***Q2: Scalability with model parameters****:* Figure 4 reports end-to-end verification time as a function of the segment budget. The one-time dynamic-programming abstraction overhead is modest: 0.15s, 0.43s, 1.006s, and 1.82s for the networks respectively. This preprocessing phase scales sublinearly with model size because spline approximation and minimax DP construction are fully parallelizable across spline components. Consequently, the abstraction overhead becomes negligible when amortized across multiple verification queries.

For the *Small* benchmark, the optimized verifier achieves a consistent 1.6–3.3× runtime improvement over uniform allocation across all budgets. The speedup originates from the sparsity induced by adaptive allocation: low-sensitivity splines receive few breakpoints, substantially reducing the size of the downstream MILP encoding. The effect becomes significantly more pronounced on the *Large* benchmark (100 inputs, 1010 parameters). At $k = 50$, the optimized verifier solves in 0.09s compared to 1.46s for the uniform baseline, corresponding to a 15.1× speedup.

This constitutes the principal scalability result of the framework. Under uniform allocation, the MILP formulation grows proportionally to $|\mathcal{S}| \times k$, yielding up to 5000 PWL breakpoints for the *Large* benchmark at $k = 50$. Such uniformly dense encodings substantially increase branch-and-bound complexity and weaken solver pruning efficiency. In contrast, the optimized formulation concentrates representational complexity only along verification-critical spline regions.

The *Medium, Medium-2* benchmarks again exhibits non-monotone behavior, with runtime ratios varying as expected. Although the optimized allocation may yield tighter bounds, the induced breakpoint structure can occasionally generate branch-and-bound trees that are harder to prune than those produced by uniform allocation. This suggests that minimizing propagated approximation error alone is insufficient as an allocation objective and motivates future formulations that jointly optimize abstraction tightness and MILP tractability.

VIII. Conclusion

The conclusion goes here.

Acknowledgment

The authors would like to thank...

REFERENCES

[1] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljacic, T. Y. Hou, and M. Tegmark, "KAN: Kolmogorov–arnold networks," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=Ozo7qJ5vZi

[2] A. N. Kolmogorov, "On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition," *Doklady Akademii Nauk SSSR*, vol. 114, no. 5, pp. 953–956, 1957.

[3] V. Arnold, "On the representation of functions of several variables by superpositions of functions of fewer variables," *Matematicheskoe Prosveshchenie*, vol. 3, pp. 41–61, 1958.

[4] D. A. Sprecher, "A numerical implementation of kolmogorov's superpositions," *Neural Networks*, vol. 9, no. 5, pp. 765–772, 1996.

[5] S. S. Bhattacharjee, "TorchKAN: Simplified KAN model with variations," https://github.com/1ssb/torchkan/, 2024.

[6] A. A. Afzal, "rkan: Rational kolmogorov-arnold networks," 2024. [Online]. Available: https://arxiv.org/abs/2406.14495

[7] S. SS, K. AR, G. R, and A. KP, "Chebyshev polynomial-based kolmogorov-arnold networks: An efficient architecture for nonlinear function approximation," 2024. [Online]. Available: https://arxiv.org/abs/2405.07200

[8] A. S. Chen, "Gaussian process kolmogorov-arnold networks," 2024. [Online]. Available: https://arxiv.org/abs/2407.18397

[9] J. Xu, Z. Chen, J. Li, S. Yang, W. Wang, X. Hu, and E. C. H. Ngai, "Fourierkan-gcf: Fourier kolmogorov-arnold network – an effective and efficient feature transformation for graph collaborative filtering," 2024. [Online]. Available: https://arxiv.org/abs/2406.01034

[10] R. Bellman, "On the approximation of curves by line segments using dynamic programming," *Commun. ACM*, vol. 4, no. 6, pp. 284–284, 1961.

[11] R. Bellman and R. Roth, "Curve fitting by segmented straight lines," *J. Amer. Statist. Assoc.*, vol. 64, pp. 1079–1084, 1969.

[12] H. Kellerer, U. Pferschy, and D. Pisinger, "The multiple-choice knapsack problem," in *Knapsack Problems*. Springer, Berlin, Heidelberg, 2004. [Online]. Available: https://doi.org/10.1007/978-3-540-24777-7_11

[13] H. P. Williams, *Model Building in Mathematical Programming*, 5th ed. John Wiley & Sons, 2013.

[14] Z. Li, "Kolmogorov-arnold networks are radial basis function networks," *arXiv preprint arXiv:2405.06721*, 2024.

APPENDIX A
DISCRETIZATION ERROR

Theorem 4. *Consider any interval* $[\mathsf{n}(j_1), \mathsf{n}(j_2)]$ *for* $0 \leq j_1 < j_2 \leq j_{\max}$ *and let* e_{j_1,j_2} *denote the error between* ψ *and* $\widehat{\psi}$ *over all the discretization points in the interval* $[\mathsf{n}(j_1), \mathsf{n}(j_2)]$*. Also, let* $c_{\max}$ *be the maximum absolute value of the slope of any piece of* $\widehat{\psi}$ *over the interval*

$$e_{j_1,j_2} \leq \max_{z \in [\mathsf{n}(j_1), \mathsf{n}(j_2)]} |\psi(z) - \widehat{\psi}(z)| \leq e_{j_1,j_2} + (c_{\max} + \psi'_{\max})\Delta$$

wherein $\psi'_{\max} = \max_{z \in [\mathsf{n}(j_1), \mathsf{n}(j_2)]} \left| \frac{d\psi}{dz} \right|$.

We start by proving a useful lemma that derives an error bound for an interval between two consecutive grid points $[\mathsf{n}(j), \mathsf{n}(j+1)]$ for $1 \leq j < j_{\max}$.

Lemma 2. *Let* $e_j = |\psi(\mathsf{n}(j)) - \widehat{\psi}(\mathsf{n}(j))|$ *for some index* $j \in [1, j_{\max})$*. For any* $z \in [\mathsf{n}(j), \mathsf{n}(j+1)]$*, the error* $|\psi(z) - \widehat{\psi}(z)| \leq e_j + (\psi'_{\max} + |c|)\,\Delta$*, wherein* $\psi'_{\max} = \max_{z \in [\mathsf{n}(j), \mathsf{n}(j+1))} \left| \frac{d\psi}{dz} \right|$ *and* $\widehat{\psi}(z) = cz + d$ *for* $z \in [\mathsf{n}(j), \mathsf{n}(j+1)]$

Proof. Consider any $z \in [\mathsf{n}(j), \mathsf{n}(j+1)]$. Note that by the mean value theorem: $\psi(z) = \psi(\mathsf{n}(j)) + \psi'(z^*)(z - \mathsf{n}(j))$ for some $z^* \in [\mathsf{n}(j), z]$. Therefore, we have, $|\psi(z) - \psi(\mathsf{n}(j))| \leq |\psi'(z^*)| \times |z - \mathsf{n}(j)| \leq \psi'_{\max}\Delta$ which leads us to:

$$\begin{aligned}
|\psi(z) - \widehat{\psi}(z)| &\leq |\psi(z) - \psi(\mathsf{n}(j)) + \psi(\mathsf{n}(j) - \widehat{\psi}(\mathsf{n}(j)) + \widehat{\psi}(\mathsf{n}(j)) - \widehat{\psi}(z)| \\
&\leq |\psi(z) - \psi(\mathsf{n}(j))| + |\psi(\mathsf{n}(j) - \widehat{\psi}(\mathsf{n}(j))| + |\widehat{\psi}(\mathsf{n}(j)) - \widehat{\psi}(z)| \\
&\leq \psi'_{\max}\Delta + e_j + |c(z - \mathsf{n}(j))| \quad \leq \quad e_k + (\psi'_{\max} + |c|)\,\Delta
\end{aligned}$$

□

Note that $\psi'_{\max}$ can be computed using the function DERIVATIVEINTERVAL assumed in Assumption 1. Now, we can use Lemma 2 to derive a correction for singlePieceError using the following theorem:

Theorem 4. *Consider any interval* $[\mathsf{n}(j_1), \mathsf{n}(j_2)]$ *for* $0 \leq j_1 < j_2 \leq j_{\max}$ *and let* e_{j_1,j_2} *denote the error between* ψ *and* $\widehat{\psi}$ *over all the discretization points in the interval* $[\mathsf{n}(j_1), \mathsf{n}(j_2)]$*. Also, let* $c_{\max}$ *be the maximum absolute value of the slope of any piece of* $\widehat{\psi}$ *over the interval*

$$e_{j_1,j_2} \leq \max_{z \in [\mathsf{n}(j_1), \mathsf{n}(j_2)]} |\psi(z) - \widehat{\psi}(z)| \leq e_{j_1,j_2} + (c_{\max} + \psi'_{\max})\Delta$$

wherein $\psi'_{\max} = \max_{z \in [\mathsf{n}(j_1), \mathsf{n}(j_2)]} \left| \frac{d\psi}{dz} \right|$.

Proof. Follows directly from applying Lemma 2 over interval $[\mathsf{n}(j), \mathsf{n}(j+1)]$ for $j_1 \leq j < j_2$. □

APPENDIX B
PROOF OF THEOREM 1

Theorem 1. *For each unit* $\psi^{(i)}_{j,k}$ *there exists a constant* $W^{(i)}_{j,k} \geq 0$ *such that the overall approximation error is* $|y - \hat{y}| \leq \sum_{i=1}^{K} \sum_{j=1}^{n_{i+1}} \sum_{k=1}^{n_i} W^{(i)}_{j,k} \times \mathsf{e}^{(i)}_{j,k}$.

Furthermore, for any i, j, k*, the constant* $W^{(i)}_{j,k}$ *is:*

$$W^{(i)}_{j,k} = \sum_{\pi \in \Pi^{(i)}_{j,k}} \prod_{\psi^{(i')}_{j',k'} \in \pi} \mathsf{L}^{(i')}_{j',k'}\text{, wherein,}$$

$\Pi^{(i)}_{j,k}$ *denotes the set of all paths from the output of the unit* $\psi^{(i)}_{j,k}$ *to the output of the KAN. The constant* $W^{(i)}_{j,k}$ *is the sum over all paths of the product of the Lipschitz constants* $\mathsf{L}^{(i')}_{j',k'}$ *(from Lemma 1) of the units encountered along each path.*

Proof. Consider the network to be a Directed Acyclic Graph (DAG) $G = (V, E)$, where, V represents two kinds of units (a) set of all KAN univariate functions and (b) summation operators, and E represents the connections between these nodes. Let $\psi^i_{j,k} = y$ denote the KAN unit and $\hat{\psi}^i_{j,k} = \hat{y}$ represent the PWA approximation of the unit incurring some error $\mathsf{e}^i_{j,k}$.

Let the layers be represented as $\{0, 1, \ldots, K\}$, where layer 0 is the input and layer i has n_i units $\{\psi^i_{j,k}\}_{j=1,\ldots,n_{i+1}}$ receiving inputs from layer $i-1$.

a) Base Case (i=1) : At layer 1, each unit $\psi^1_{j,k}$ takes input z from the network and by Lemma 1 we have $|\psi^1_{j,k} - \hat{\psi}^1_{j,k}| \leq \mathsf{e}^1_{j,k} + \mathsf{L}^1_{j,k}|z - \hat{z}|$. However, since we know that z and $\hat{z}$ are the same actual inputs we have:

$$\begin{aligned}|\psi^1_{j,k} - \hat{\psi}^1_{j,k}| &\leq \mathsf{e}^1_{j,k} + \mathsf{L}^1_{j,k}|z - \hat{z}| \\ &\leq \sum_{j=1}^{n_{i+1}} \sum_{k=1}^{n_i} W^1_{j,k} \mathsf{e}^1_{j,k} \\ &\leq \sum_{i=1} \sum_{j=1}^{n_{i+1}} \sum_{k=1}^{n_i} W^i_{j,k} \mathsf{e}^i_{j,k}\end{aligned}$$

where, $W^1_{j,k} = 1$ since there is exactly one path of length zero from $\psi^1_{j,k}$ to itself.

b) Inductive Proof: Assume that for every layer $k-1$ we have:

$$\sum_{i=1}^{k-1} \sum_{j=1}^{n_{i+1}} \sum_{k=1}^{n_i} W^i_{j,k} \mathsf{e}^i_{j,k}$$

Consider a unit $\psi^i_{j,k}$ in layer i. The activations satisfy

$$\begin{aligned}z^i_{j,k} &= \sum_{(\ell,m)\to(j,k)} w^i_{j,k;\,\ell,m}\, \psi^{i-1}_{\ell,m}(z^{i-1}_{\ell,m}) \\ \hat{z}^i_{j,k} &= \sum_{(\ell,m)\to(j,k)} w^i_{j,k;\,\ell,m}\, \hat{\psi}^{i-1}_{\ell,m}(\hat{z}^{i-1}_{\ell,m})\end{aligned}$$

Hence, we get

$$\left|z^i_{j,k} - \hat{z}^i_{j,k}\right| \leq \sum_{(\ell,m)\to(j,k)} \left|w^i_{j,k;\,\ell,m}\right| \, |\psi^{i-1}_{j,k} - \hat{\psi}^{i-1}_{j,k}|$$

Applying Lemma 1 again gives

$$\begin{aligned}|\psi^i_{j,k} - \hat{\psi}^i_{j,k}| &\leq \mathsf{e}^i_{j,k} + \mathsf{L}^i_{j,k} \left|z^i_{j,k} - \hat{z}^i_{j,k}\right| \\ &\leq \mathsf{e}^i_{j,k} + \mathsf{L}^i_{j,k} \sum_{(\ell,m)\to(j,k)} \left|w^i_{j,k;\,\ell,m}\right| \, |\psi^{i-1}_{j,k} - \hat{\psi}^{i-1}_{j,k}|\end{aligned}$$

By substituting the inductive bounds, the $\mathsf{e}^i_{j,k}$ term in those bounds is multiplied by an extra factor $\mathsf{L}^i_{j,k}\,|w^i_{j,k;\,\ell,m}|$, which accounts for extending existing paths by one more edge and node. Collecting terms givs us:

$$|\psi^i_{j,k} - \hat{\psi}^i_{j,k}| \leq \sum_{i=1}^{k} \sum_{j=1}^{n_{i+1}} \sum_{k=1}^{n_i} W^i_{j,k}\, \mathsf{e}^i_{j,k}$$

with each new $W^i_{j,k}$ obtained by summing (over all paths π reaching layer i the product of the L–constants along π. Conceretly, if $\pi^i_{j,k}$ are the old paths stopping at layer $i-1$, then

$$\prod_{\psi^{(i')}_{j',k'} \in \pi} \mathsf{L}^{(i')}_{j',k'}$$

enumerates the extended paths along the edges. Finally, taking $i = K$ gives the final output error $|y - \hat{y}|$ which can be equated to

$$|y - \hat{y}| \leq \sum_{i=1}^{K} \sum_{j=1}^{n_{i+1}} \sum_{k=1}^{n_i} W^i_{j,k} \mathsf{e}^i_{j,k}$$

□

APPENDIX C
PROOF OF THEOREM 2

Theorem 2. *The (decision version) of the multi-option knapsack problem is **NP**-complete.*

Proof. The decision Multi-Option Knapsack (MOK) problem can be shown to be **NP**-complete by a reduction from the classical $0-1$ knapsack problem, which is known to be **NP**-complete (which in turn reduced from the subset-sum problem).

Given a candidate solution for the decision version of Problem 3 we can perform the following checks i.e. (a) does the total weight of the selected items within the budget specified W? (b) does the total value of all items account to at least the target

value V?, and (c) is exactly one item is selected from each option? . We can verify all these conditions in $O(Nk)$ polynomial time, leading us to MOK $\in$ **NP**.

Now, we can reduce from the decision version of the $0-1$ knapsack problem, where we can set the problem instance as selecting a set of $N = m$ items each with weight $w_j \in \mathbb{N}$, value $v_j \in \mathbb{N}$, a weight bound $W \in \mathbb{N}$, and a maximum target value $V \in \mathbb{N}$. We need to find out: Is there a subset $S \subseteq \{1, \ldots, m\}$ such that $\sum_{j \in S} w_j \leq W$ and $\sum_{j \in S} v_j \geq V$?

Given such an instance, we can formulate the problem where, for each item i in the $0-1$ knapsack problem, we can define an option k with two items:

1) Item $I^k_{i,1}$ with weight $w^k_{i,1} = 0$ and value $v^k_{i,1} = 0$, corresponding to not selecting the item j
2) Item $I^k_{i,2}$ with weight $w^k_{i,2} = w_j$ and value $v^k_{i,2} = v_j$, corresponding to selecting item j

If the knapsack problem has a subset S satisfying the constraints of the MOK problem, then for each i choosing item $I^i_{i,2}$ if $j \in S$ and $I^i_{i,1}$ otherwise, will lead to a valid MOK solution with total weight $\leq W$ and value $\geq V$. Conversely, any feasible solution that corresponds to a selection of items where each option contributes to either no weight and value or weight and value corresponding to that item selected. In that case, the set of indices j where $I^j_{j,2}$ is selected forms a feasible solution to the $0-1$ knapsack problem.

It is clear that this process of converting the problem is polynomial time in number of inputs N. Thus, we have shown that $0-1$ knapsack $\leq_p$ MOK. □

APPENDIX D
ESTIMATION OF M-CONSTANTS

We will explain how the bounds M_z and M_y are calculated. For each unit, the bound M_z is calculated based on one of two cases:

1) If the unit's input is that of the entire KAN, its range is known as part of the problem input and thus we can compute M_z directly from the input bound.
2) If the unit's input is a sum node that involves the outputs of k units, the value of M_z for the unit is simply the sum of the value of M_y's for the units whose outputs connect to the input through the summation node.

To compute M_y for a given KAN, we recall the linearized function

$$\widehat{\psi}(z) = \begin{cases} 0 & z \leq -L \\ a_i z + b_i & z \in [z_i, z_{i+1}],\ i \in \{0, \ldots, \ell - 1\} \\ 0 & z \geq L \end{cases}$$

Note that for all z, $|\widehat{\psi}(z)| \leq \max_{i=0}^{\ell-1} \max(0, |a_i z_i + b_i|, |a_i z_{i+1} + b_i|)$. Let $\hat{M}_y$ be the maximum value for $|\widehat{\psi}(z)|$. Therefore, $M_y = \max_z |\psi(z)| \leq \hat{M}_y + |e|$.

APPENDIX E
MORE KAN BENCHMARKS

We present the graphs of the results from rest of the benchmarks that were used in the original KAN paper [1] which we did not present in the main paper due to space constraints. The five tasks are as follows:

1) *Small*: `funcbessel`: $f(x) = \mathcal{J}_0(20x)$ (1 inputs; 20 parameters; [1, 1] KAN)
2) *Small*: `funcellipeinc`: $f(x_1, x_2) = E(x_1 \,|\, x_2)$, where $E(\phi \,|\, m) = \int_0^\phi \sqrt{1 - m \sin^2 \vartheta}\, d\vartheta$ (2 inputs; 70 parameters; [2, 2, 1, 1] KAN)
3) *Medium*: `funcellipkinc`: $f(x_1, x_2) = K(x_1 \,|\, x_2)$, where $K(\phi \,|\, m) = \int_0^\phi \frac{d\vartheta}{\sqrt{1 - m \sin^2 \vartheta}}$ (2 inputs; 70 parameters; [2, 2, 1, 1] KAN)
4) *Small*: `funclegendre`: $f(x) = P^m_\nu(x_1)$. For integer $m \geq 0$, one standard definition is $P^m_\nu(x) = (1 - x^2)^{m/2} \frac{d^m}{dx^m} P_\nu(x)$, where P_ν is the (degree-ν) Legendre function of the first kind (1 input; 70 parameters; [1, 100, 1] KAN)
5) *Small*: `funcsphharm`: $f(x_1, x_2) = \Re\{Y^m_n(x_1, x_2)\}$. Using $\theta = x_1$ (azimuth) and $\phi = x_2$ (polar), $Y^m_n(\theta, \phi) = \sqrt{\frac{2n+1}{4\pi} \frac{(n-m)!}{(n+m)!}}\, P^m_n(\cos \phi)\, e^{im\theta}$ (2 inputs; 140 parameters; [2, 3, 2, 1] KAN)

The results for the rest of the benchmarks are shown in Figures 6 and 7. We see that our method (*Optimized*) consistently achieves tighter bounds than the baseline (*Vanilla*) across all segment counts and is also less computationally expensive.

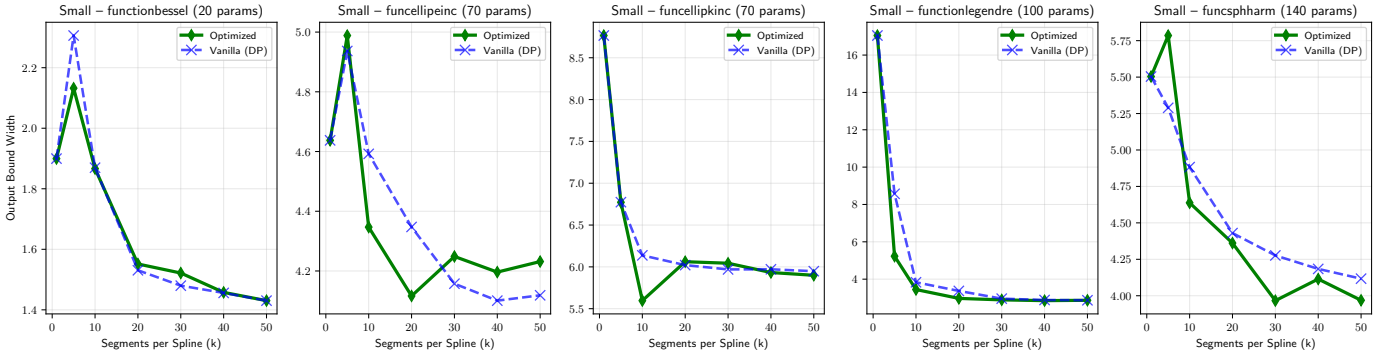

Fig. 5: Tightness of the methods compared by showing how the output bound width changes as segment count per spline is varied. We see that across all methods, the output bound width increases as the segment count increases. Our method (*Optimized*) consistently achieves tighter bounds than the baseline (*Vanilla*) across all segment counts.

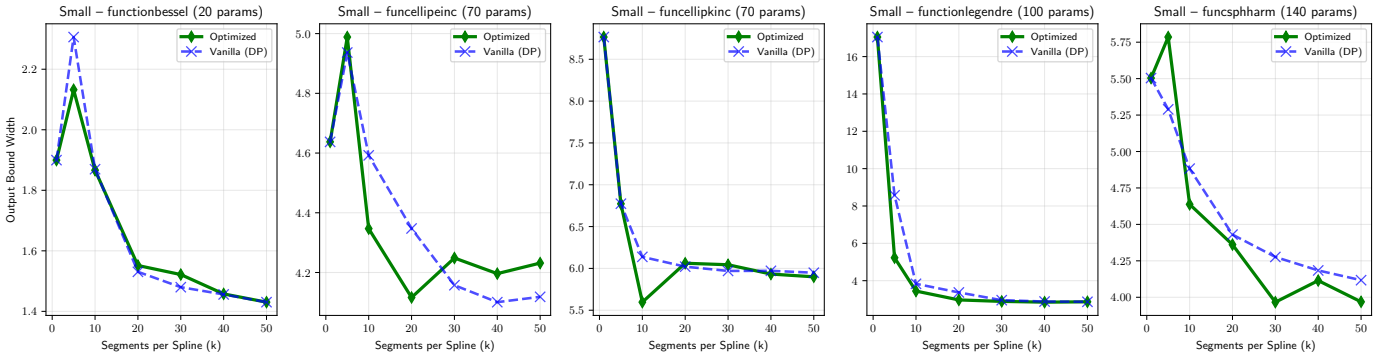

Fig. 6: Tightness of the methods compared by showing how the output bound width changes as segment count per spline is varied. We see that across all methods, the output bound width increases as the segment count increases. Our method (*Optimized*) consistently achieves tighter bounds than the baseline (*Vanilla*) across all segment counts.

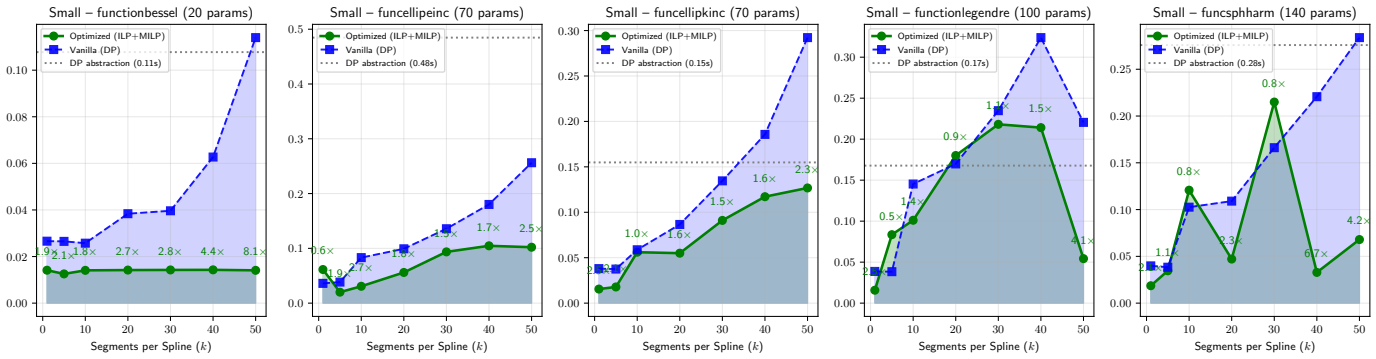

Fig. 7: Timing summary for the different methods. We see that our method (*Optimized*) is less computationally expensive than the baseline (*Vanilla*) and also achieves tighter bounds almost across all segment counts for the models.